

CANNBot开发进阶： Vector算子之排序性能优化

作者：马钺迪，高雪健，郑灵超，李君，樊玉伟，杨继伟，黄强

时间：2026-05-18

现在，开始您的CANNBot之旅~ 让高效算子开发触手可及^_^

<https://gitcode.com/cann/cannbot-skills>

CANNBot代码仓库URL，欢迎
star、fork、download、PR



请扫码加入 “CANNBot Skills开
源交流2群”

群聊：CANNBot Skills开源交流
2群



“CANNBot Skills开源交流1群”，
已超过200人，不能扫码，可通过好
友邀请入群。

CANN

该二维码7天内(5月22日前)有效，重新进入将更新

目录

Part 1 大规模排序 / 筛选问题

Part 2 CANNBot之排序性能优化

Part 3 性能优化案例实践

什么是大规模排序 / 筛选

核心场景

在 N 个元素中找出满足某种顺序和条件的 K 个，且 N 或 K 很大、数据放不进单核片上存储。

常见形态：

- **Top-K**：选最大 / 最小的若干个
- **Bottom-K / Percentile**：分位数
- **Top-K + Index**：返回元素 + 原始位置
- **Sort**：全排序 ($K = N$)
- **Filter / Threshold**：满足条件的全部

问题特征：

- 数据量远超片上存储
- 跨核 / 跨卡需协调
- 调用频繁，对延迟敏感
- 在 LLM / 推荐系统等场景中存在特殊数据分布

一些典型应用场景：

- **DeepSeek V3.2 DSA / V4 CSA — LightningIndexer 中的 Top-K**
 - 稀疏注意力：每 query 用 LightningIndexer 给所有 key 打分，选出最相关的若干个 key 做 attention 计算
 - Top-K 计算部分无法被流水掩盖，成为 LightningIndexer 部分的瓶颈
- **推荐系统多级排序漏斗**
 - 召回 → 粗排 → 精排：逐级缩小，每级需要大规模排序/筛选
 - 召回阶段 N 极大，端到端有严格实时性要求

为什么困难

选什么算法?

Sort
 $O(N \log N)$
全排序

Bitonic Sort
GPU/NPU 友好
 $O(N \log^2 N)$

QuickSelect
期望 $O(N)$
不稳定

IntroSelect
最坏 $O(N)$
C++ std

Heap TopK
 $O(N \log K)$
 K 大时溢

Radix Select
 $O(N \cdot b)$
本工作

Binary Search
+ Counting
 $O(N \cdot \log V)$

Two-level
local + merge
CANN topk

Merge Sort

Radix Sort

Median of Med

Sample Sort

Agent 的能力

熟悉 10+ 种变体

根据 N/K/硬件/精度

匹配 sweet spot

排序类

选择类

并行/混合

算法选择多

(a) tie-break: 相等元素如何排序?

输入 (idx 标在格子里):

3.14	3.14	3.14	3.14	3.14	3.14	3.14	3.14
------	------	------	------	------	------	------	------

idx=7 idx=2 idx=19 idx=5 idx=11 idx=0 idx=14 idx=3
全部 = 3.14, 要选前 4 个, 到底取哪 4 个?
torch.topk 规则: 取 index 升序最小的若干个 ✓
应选 idx ∈ {0, 2, 3, 5}
朴素并行: 各核出 idx 任意 X
可能选 idx ∈ {7, 19, 11, 14} — 数值对, idx 错 → overlap < K
Agent 的能力: 多核之间 + 跨硬件之间 都对齐 — 否则单元测试瞬间挂掉

(b) 跨硬件 bit-exact 复现

GPU (torch.topk)
FP 比较 + index 升序

NPU 自定义算子
必须输出完全相同

CPU 参考实现
必须输出完全相同

全部 bit-exact

相等元素排序, bit-exact

数据布局
chunk / 分区

UB 容量
192 KB / core

DMA 带宽
MTE2 / MTE3

跨核同步
无硬件 barrier

流水线编排
MTE2 || Vec || Scalar

向量 vs 标量
任务划分

Cache-line 对齐
32B = 8 × int32

数值精度
FP / INT 边界、饱和

Agent 的能力

系统级地兼顾每一项约束—单点忽略就掉性能 / 出 bug

NPU
Kernel 实现

硬件细节复杂

来自哪些场景?

(1) LightningIndexer / DSA
稀疏注意力打分 → 99%+ 稀疏

(2) 推荐系统打分
长尾分布 + 分位数截断

(3) 量化 / 离散数据
int8 / 4-bit 量化 → 大量重复值

Normal $\mathcal{N}(0, 1)$
基准情况

All zeros
全部相等 ★

Sparse 99.5%
几乎全为零

Bimodal ±100
两个峰

Agent 的能力: 这些“非典型”分布上, 朴素实现往往慢 5-10×; 要写 特化 fast-path

多种特殊数据分布

这正是 Human+Agent 协作大有可为的地方

算法变体多、硬件细节多、问题 case 多 —— Agent 擅长详尽枚举和定位bug, 测试拆解性能瓶颈, Human 把握方案设计。

任务目标

■ 面向Ascend 910/Ascend 950
实现一个**大规模Top-k**算子

■ 严格 bit-exact **精度对齐**
torch.topk

■ 各种数据分布**性能稳定**

■ **Human+Agent** 的协作开发
模式

Human + Agent 协作开发

- **工程师** 提出问题与方向，分析并选取基数排序算法作为主要算法思路
- **Agent** 实现、测试、定位 bug，调通精度
- **Agent** 版本演化：每个版本逐步添加优化点并实时验证
- **Agent** 瓶颈分析，分析主要耗时瓶颈
- **工程师** 基于瓶颈分析调整算法：算法从朴素基数选择 → 流水线 & 极端分布特化的工业级实现

目录

Part 1 大规模排序 / 筛选问题

Part 2 CANNBot之排序性能优化

Part 3 性能优化案例实践

我们给 CANNBot 提供了什么

硬件&框架文档

- 昇腾硬件基础知识
- UB / GM 容量与对齐约束
- 三流水线 (MTE2 / Vector / Scalar / MTE3) 模型
- Ascend C 编程接口 (TQue、TBuf、DataCopy、向量指令)

测试&性能

- 覆盖常见 + 极端分布的数据生成器
- bit-exact 对齐脚本 (VS torch.topk)
- msProf 性能采集
- 冷启动 VS warmup benchmark

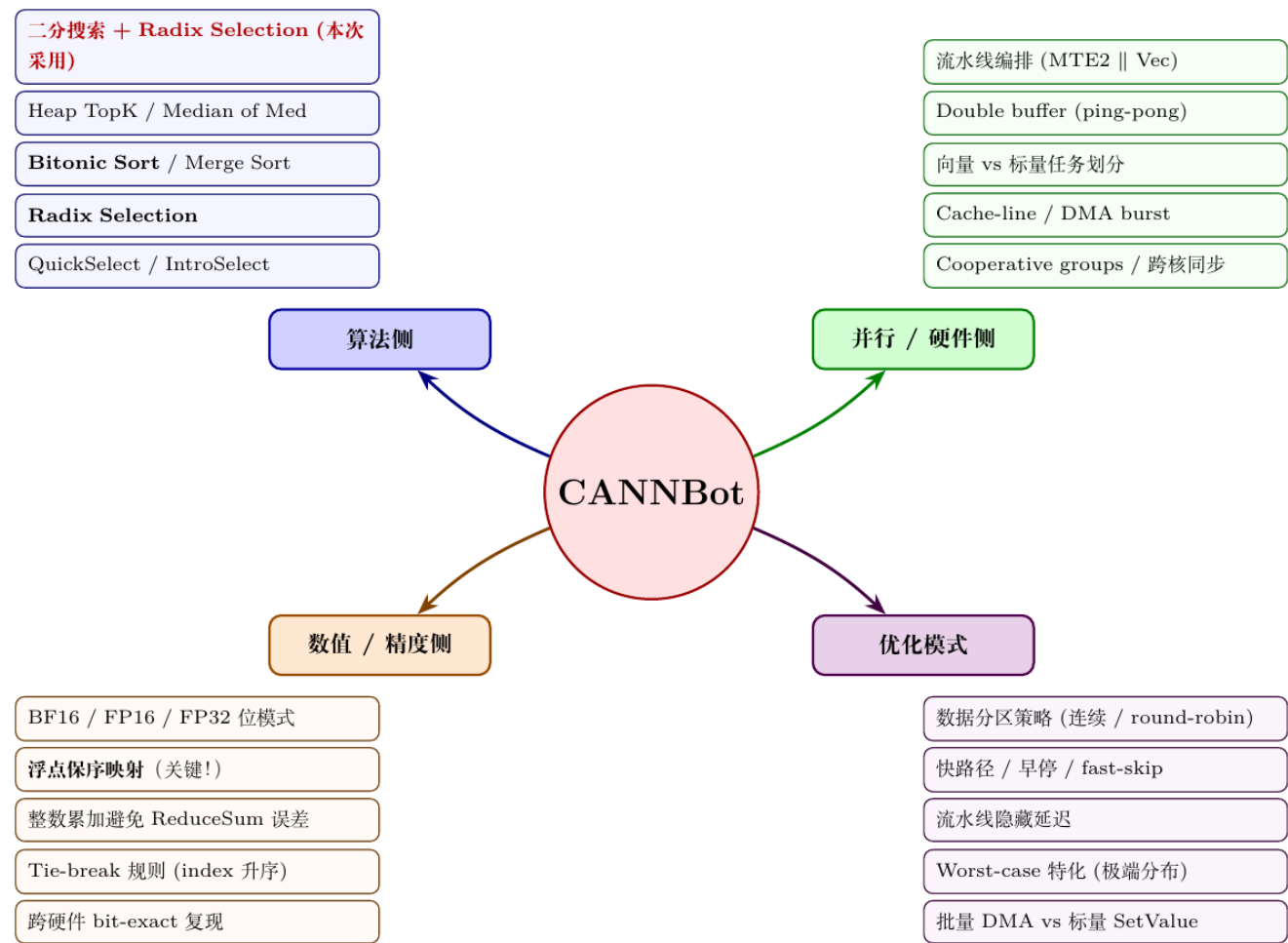
算子骨架 & 工具链

- host tiling 模板
- kernel 骨架 + CMake
- build_and_install.sh

历史 & 上下文

- 历史版本的 Release Notes, 记录
- 踩坑经验 (保序映射、Adds 饱和等)
- Skills: 关于排序/筛选类任务的知识

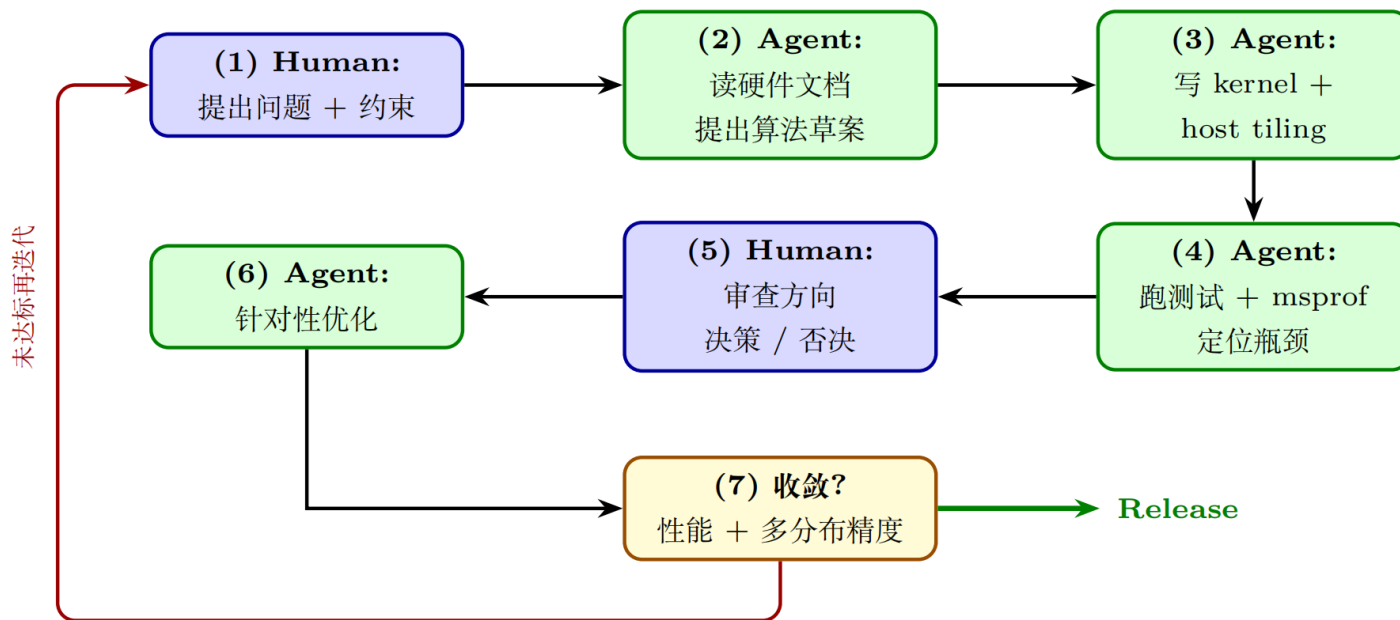
CANNBot 关于排序 / 筛选类问题的知识



抽取当前任务相关的skill

- 算法类skill
- 并行/硬件类skill
- 数值/精度类skill
- 优化类skill

Human + Agent 开发流程



关键观察

- **Agent 擅长：** 算法变体探索、详尽测试、定位硬件级 bug (Agent 反向构造测试)
- **Human 擅长：** 算法分析和设计、方向决策、权衡判断
- 这种模式相比传统单人开发，迭代节奏明显加快

目录

Part 1 大规模排序 / 筛选问题

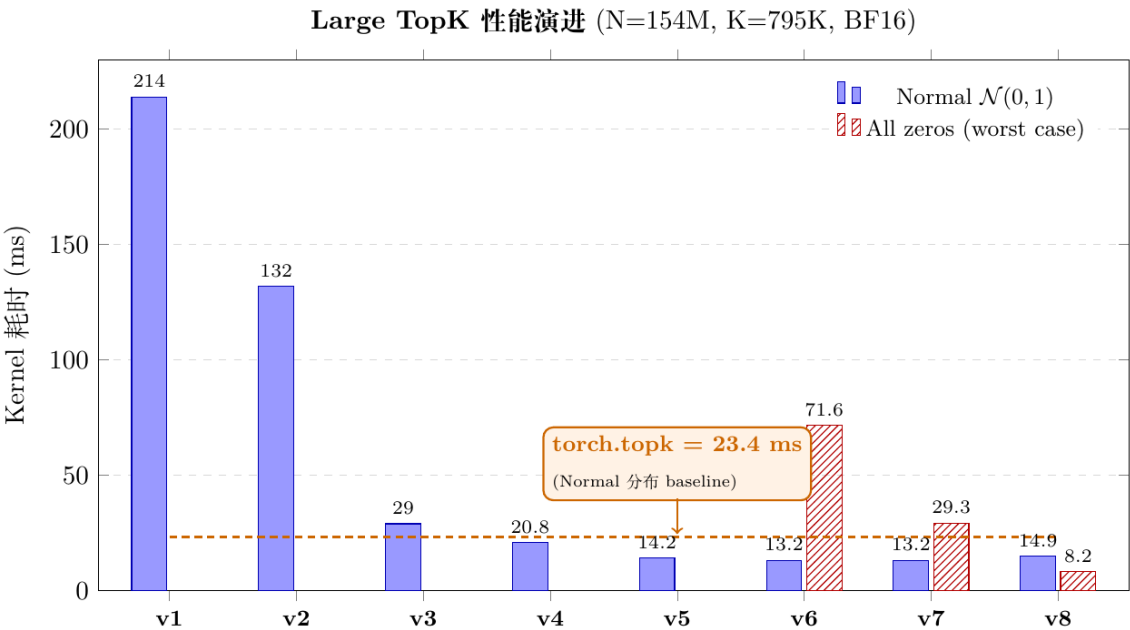
Part 2 CANNBot之排序性能优化

Part 3 性能优化案例实践

案例实践：Large Top-K on Ascend 910

问题定义

- 输入: $x \in \text{BF16}[154,256,400]$
- 输出: $K = 794,880$ 个最大值 + index
- 硬件: Ascend 910 (48 AIV cores, UB 192 KB/core)
- 精度: bit-exact 对齐 torch.topk



- **K 非常大 (80 万)**
 - Heap-based TopK 需 $K \times 2 \text{ Byte} = 1.6 \text{ MB UB}$, 无法容纳
 - Sort 后取 K 个 $O(N \log N)$, 太慢
 - $\Rightarrow$ 用 **Selection (按阈值筛)** 思路
- **N 远大于 UB**
 - 数据必须在 GM 多次扫描, DMA 带宽 Bound
 - 跨核同步只能 GM 轮询 (无硬件 barrier)
- **BF16 比较**: 浮点位模式不能按 uint16 比较 (正负数顺序相反)
- **Tie-break 物理约束**: torch.topk 要求相等元素按 index 升序
 - All zeros 时所有元素都"相等" $\rightarrow$ index 必须取最小的 K 个
 - 朴素实现下, core 0 单核成为瓶颈

Normal数据集基准对比

方法	耗时
torch.topk(unsorted)	23.4 ms
基数排序优化版 (v8)	14.9 ms

方案设计：为什么使用基数排序？

方法	复杂度	本场景可行性	主要问题
Heap-based TopK	$O(N \log K)$	✗	K=80 万，UB 装不下
Sort 后取前 K	$O(N \log N)$	✗	N=10^8, 太慢 (torch 374 ms)
Bitonic Sort + Merge	$O(N \log^2 N)$	可行	CANN 内部用，但实现极复杂
Radix Selection	$O(N \cdot b)$	✓	16-bit 值域 $\Rightarrow$ 最多 16 次扫描

Radix Selection 思路

- 不排序，只找"第 K 大元素的阈值 threshold"
- 把 16-bit sortable 值域逐步分桶，缩小 threshold 所在区间
- 找到 threshold 后，扫描输出所有 > threshold 的元素，并从 = threshold 的元素中补齐

Agent辅助性能优化

Agent辅助，从Radix Selection思路出发，基于msProf拆解性能瓶颈，逐步添加优化点。



阶段1: Radix 基础 #1 [v1] BF16 → Sortable 保序映射

问题: BF16 raw 不能直接按 uint16 比较

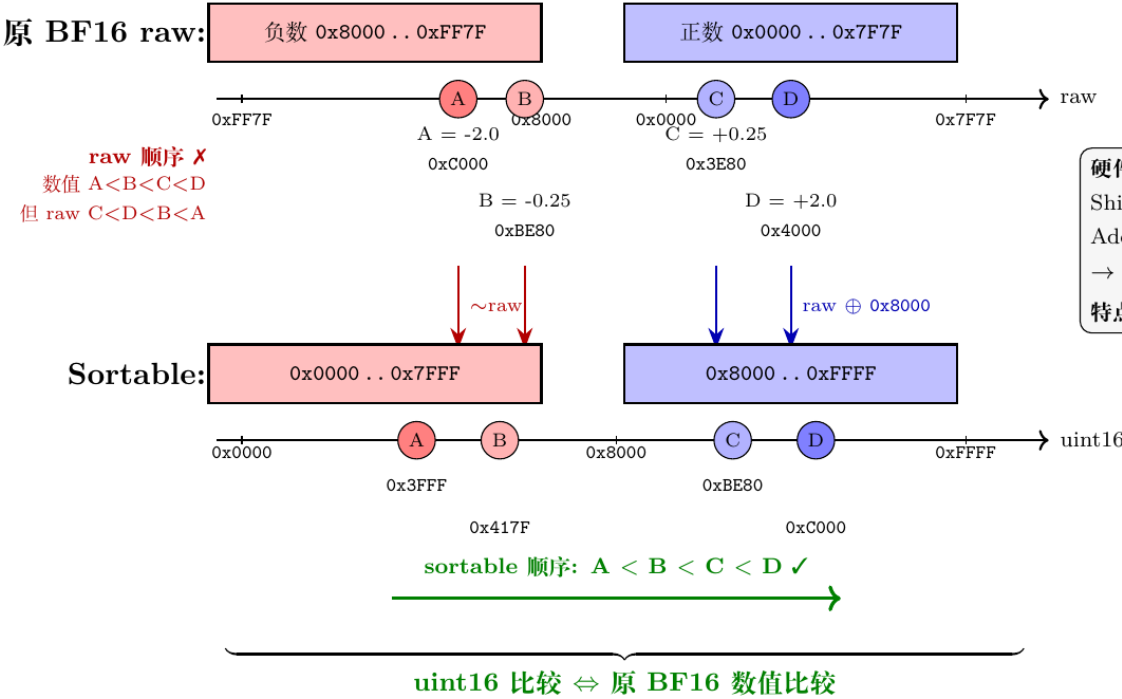
- 正数 raw 越大 -> 值越大 ✓
- 负数 raw 越大 -> 值越小 ✗

方案: 保序映射

- 正数: $\text{raw} \oplus 0x8000$
- 负数: $\sim \text{raw}$

硬件: 7条向量指令

Shr → Not → Adds(-2^{15}) → And →
Not → And → Or



硬件实现: 7 条向量指令

ShiftRight → Not →
Adds(-32768)
→ And → Not → And → Or
特点: 位精确、可逆、无浮点

保序, 可逆, 无浮点损失!

该skill可以被其他浮点排序场景使用

阶段1：Radix 基础 #2 [v3, -103 ms] 标量 16×16 Nibble 表

关键洞察：若考虑正态分布，Pass 2 阶段只有大约 $\sim 1/256$ 元素属于目标范围 $\Rightarrow$ 向量过滤是浪费

方案：标量 + XOR fast-skip

```
top8Magic = (a1_top8 << 8) | (a1_top8 << 24);  
top8Mask  = 0xFC00FC00;  
x = (word ^ top8Magic) & top8Mask;  
if ((x0 & x1 & x2 & x3) == top8Mask) continue; // 8 元素都不匹配, ~99.6% 跳过
```

- **匹配元素累积到 16×16 标量表** (整数精确, 无 Cast / ReduceSum)
- 单次扫描同时算 Pass2 L1 + L2
- **132 $\rightarrow$ 29 ms (-78%)**

稀疏问题的标量筛选

稀疏问题难以向量化处理, 标量带 skip 更合适。

阶段2：算法重构 #3 [v4, -8 ms] 3+3+5+5 分桶 + 累积 GE

改动1：分桶策略

- 4+4+4+4: 64 次 ReduceSum
- 3+3+5+5: L1/L2 各 8 bin (向量), Pass2 标量
- ReduceSum 次数减半

改动 2: 累积 GE 模式

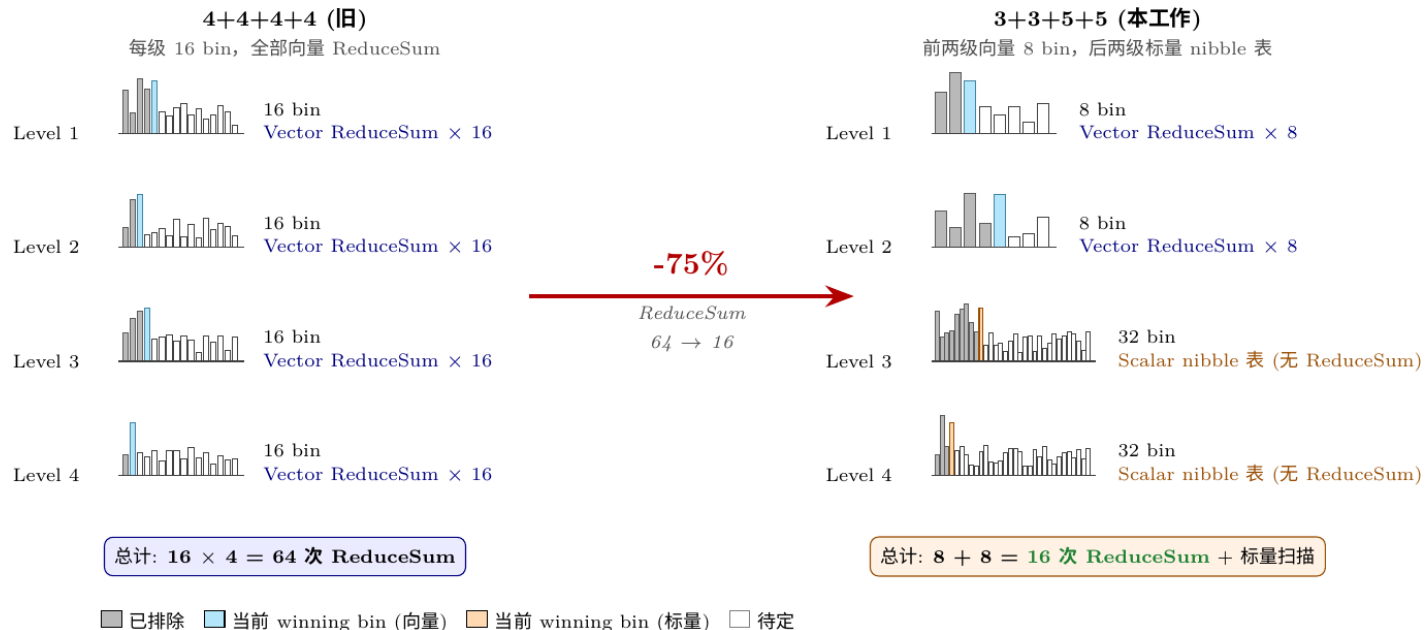
```
GE(1), GE(2), ..., GE(7) # 7 次 GE  
bin[0] = N - cum[1] # 省 1 次  
bin[k] = cum[k] - cum[k+1]
```

改动 3: 2-bin batch ReduceSum

- 每 2 GE 共享 1 个 PipeBarrier

综合收益

- ReduceSum: 64 → 30 次
- PipeBarrier 数量 -50%
- 29 → 20.8 ms



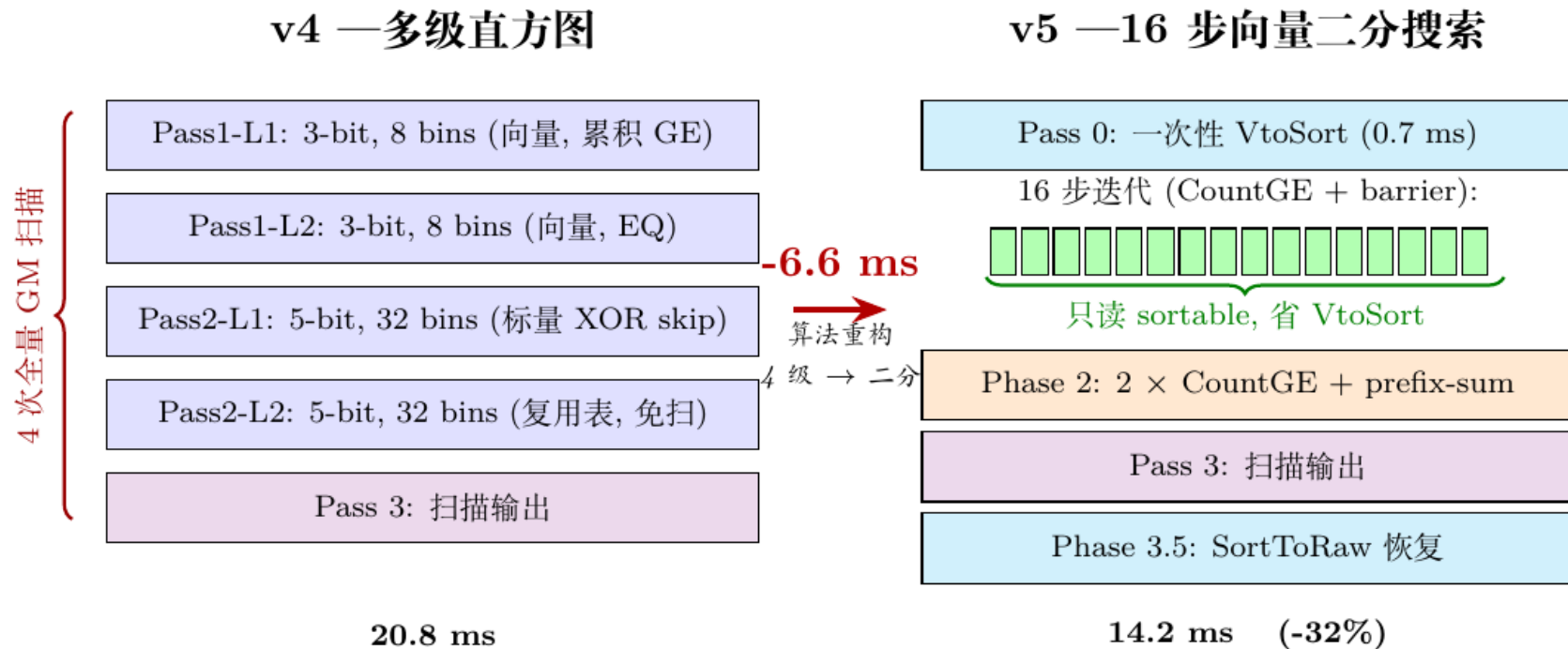
方法论

精度风险点 (v3→v4):

- Adds(int16) 跨 0x8000 饱和
- Partial subchunk padding
- Sortable/raw 域混淆

全部修复后 15/15 PASS。

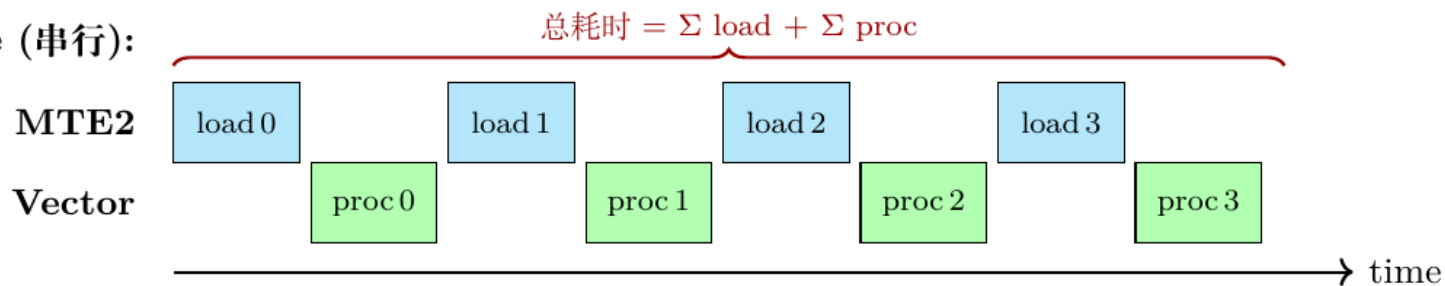
阶段2：算法重构 #4 [v5, -6.6 ms] 16 步向量二分搜索



- **核心洞察**: 4 级直方图本质做了 4 遍全数据扫描
- **重构**: 16-bit sortable 空间二分搜索, 16 步必收敛
- **Pass 0 一次性 VtoSort**: 省去后续 18 次重复 (+0.7 / -2.8 ms)

阶段3：流水线 & 工程特化 #5 [v6, -1 ms] TQue Double Buffer

Before (串行):



TQue<VECIN, 2>

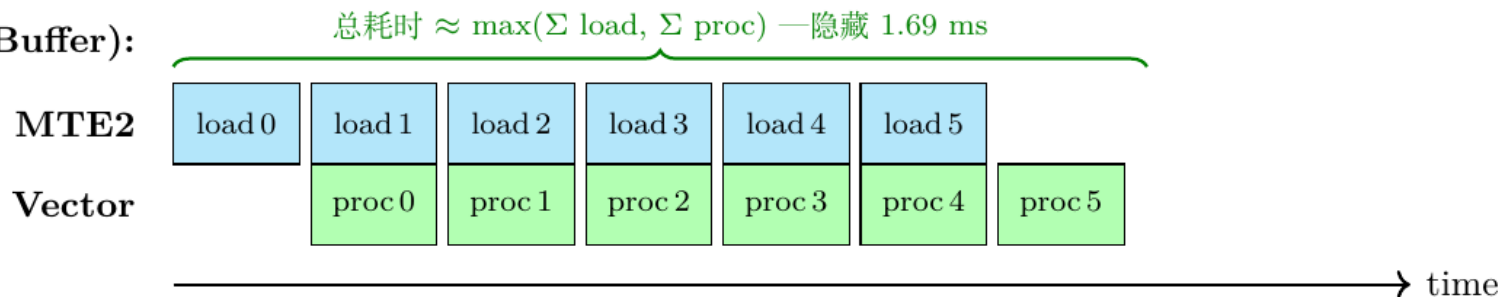
Prolog: 预加载 2 chunks

Steady: Compute(i) ||

CopyIn(i+2)

收益: -1.0 ms (v6, V6.0)

After (Double Buffer):



- TQue<VECIN, 2> — 两个 ping-pong 缓冲
- Prolog 预加载 2 chunks, steady state 中 Compute(i) 与 CopyIn(i+2) 并行
- msProf 实测: 管道并行隐藏 1.69 ms

借助了 CAKE Agent 的 Double Buffer skill

阶段3：流水线 & 工程特化 #6 [v7] Pass 3 重复数据特化

问题：v6 在 All zeros / Sparse 99.5% 等场景达 60-70 ms

- 原因：GE mask 密度 99.5%，scalar 16-group OR 快速跳过失效

三个 Pass 3 优化

- **Early stop**：above 和 equal 配额都满 → break
- **动态 useGT**：equal 配额用完后切到 GT mask (> threshold)
 - GE mask 密度 99.5% → GT mask 密度 0.25%
- **rawThresh 预计算**：equal 元素 raw 值都相等，省去 GM 读

分布	v6	v7
All zeros	71.6 ms	29.3 ms (-59%)
Sparse 99.5%	64.1 ms	23.2 ms (-64%)
Bimodal	69.6 ms	40.6 ms (-42%)

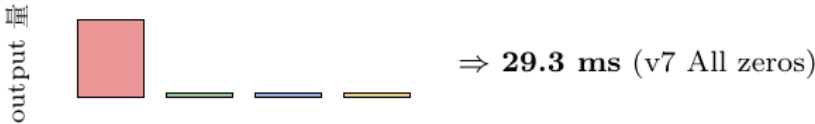
阶段3：流水线 & 工程特化 #7 [v8] chunk-level Round-robin 分区

v6 / v7 —连续分区

core c 处理 chunks $[c \cdot R, (c + 1) \cdot R)$



All zeros: 前 needEqual 个 equal index 全在 core 0 段 $\Rightarrow$ core 0 单核瓶颈



core 0

core 1

core 2

core 3 ...

equal 元素位置

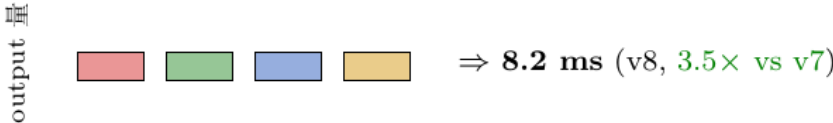
Vector 核心负载均衡!

v8 —chunk-level Round-robin

core c 处理 chunks $\{c, c + C, c + 2C, \dots\}$



All zeros: equal 散布到 48 核 $\Rightarrow$ 并行输出



代价:

round-major 顺序下相邻

ePos 间隔 $< 8 \times \text{int32}$

标量 SetValue $\rightarrow$ cache-line race

修复: 批量 DataCopyPad

借助了 CAKE Agent 的 负载均衡 skill



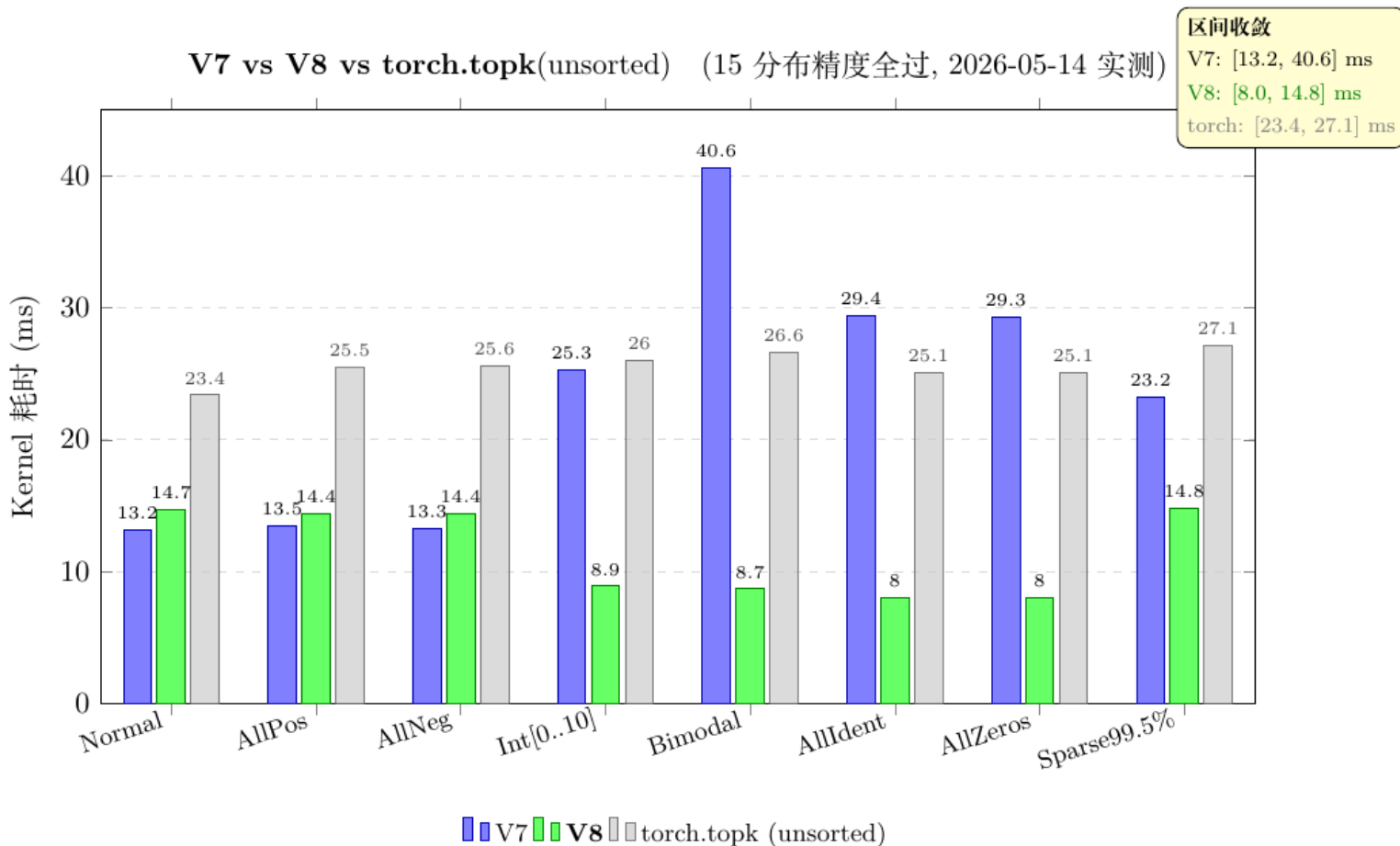
阶段3：流水线 & 工程特化 #8 [v8] 早停 + Fast-path + 动态 GT-mask

```
// 后缀和预计算 -  $O(R)$ 
sumTakesFromR[r] = Sum_{r'>=r} myChunkTakes[r'];

for (r = 0; r < R_; ++r) {
    // 整循环早停: 后面没任务 + above 已满
    if (myMaxEqual==0 && sumTakesFromR[r]==0 && aPos>=aPosEnd) break;
    // 跳过本 chunk
    if (myMaxEqual==0 && aPos>=aPosEnd) continue;
    // 全 take 快路径: 跳过 DataCopy + VtoSort + Mask
    if (myMaxEqual==bigChunk && aPos>=aPosEnd) { fast_path; continue; }
    // ... normal path
}
```

- **动态 GT-mask:** myMaxEqual == 0 时改用 > threshold
- **All zeros:** 61.5 ms → **8.3 ms (7.4×)**
- **Sparse 99.5%:** 34.0 ms → **15.0 ms (2.3×)**

最终结果：15 种分布精度全 PASS，性能提升明显



在各种分布下实现性能全面提升

现在，开始您的CANNBot之旅~ 让高效算子开发触手可及^_^

<https://gitcode.com/cann/cannbot-skills>

CANNBot代码仓库URL，欢迎
star、fork、download、PR



请扫码加入 “CANNBot Skills开
源交流2群”

群聊：CANNBot Skills开源交流
2群



“CANNBot Skills开源交流1群”，
已超过200人，不能扫码，可通过好
友邀请入群。

CANN

该二维码7天内(5月22日前)有效，重新进入将更新

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯